

Three or More Dimensional Sensor-Based Path-Planning Algorithm *HD - I*

Hiroshi Noborio⁺, Yasuhiro Maeda⁺⁺, and Kenji Urakawa⁺

⁺ Graduate School of Engineering, Osaka Electro-Communication Univ.

⁺⁺ Department of Eng. Informatics, Osaka Electro-Communication Univ.

Abstract

To design an on-line or sensor-based path-planning algorithm, we should decrease its search cost (the number N of visited points of a robot in a known map or the sum S of moving distances of it in an uncertain map). For this purpose, the optimal algorithm, e.g. an A^ algorithm is not suitable. Because in order to select an optimal path between start and goal points, the algorithm requests a huge set of visited points or driving distances. On the observation, by decreasing the sum S , we design a new sensor-based path-planning algorithm *HD - I*.*

*In almost all the sensor-based algorithms, e.g. Class1, convergence of a robot to its goal is ensured by decreasing their distance. Also in a famous algorithm Best - First, the convergence is also kept by visiting a node whose distance to its goal is the smallest. They are similar because a robot does not take a distance from a start into account. Therefore, their modified versions can be mixed. Especially, BF can investigate around an unknown obstacle whose dimensions are over two. As a result, by mixing modified BF and Class1, we obtain three or more dimensional algorithm *HD - I*.*

1 Introduction

The robot path-planning is covered by three types of algorithms, that is, off-line (called model-based) algorithm, on-line algorithm, and sensor-based algorithm. First of all, the former two algorithms run in a known map. The difference of them is whether the number N of searching points is large or small. An algorithm is regarded as the off-line algorithm if its N is large, otherwise, it is considered as the on-line algorithm. For example, in order to select an optimal (shortest) path between start and goal points, an A^* algorithm investigates a huge set of points in a known map. Therefore, A^* is to be the off-line (model-based) path-planning algorithm.

Secondly, the latter algorithm runs in an unknown map. Thus, a robot finds a path until a destination while investigating a part of the unknown map by sensing. Therefore, whether an algorithm is the sensor-based algorithm or not is evaluated by the sum S of moving distances of a robot. If and only if S is small enough, the algorithm is used for the sensor-based algorithm. Since S is exactly larger than N , an off-line (model-based) algorithm is not absolutely to be a sensor-based algorithm, and also an on-line algorithm is not always to be it.

In the past decade, many sensor-based path-planning algorithms have been proposed [1]. In the sensor-based path-planning, a robot always selects one of two behaviors, that is, going straight to its destination, and avoiding an encountered obstacle. The behaviors are switched if a robot hits or leaves an unknown obstacle. Here, as long as leave points monotonously or asymptotically come close to a destination, convergence of a robot to its destination is completely kept in an uncertain environment. This is called a metric condition. In general, a leave point is always around an encountered obstacle. Therefore, if the dimension of an obstacle is two, a robot always reaches the leave point while tracing an uncertain obstacle by one of the clockwise and counter-clockwise directions. However, the characteristic cannot be unfortunately maintained on a three or more dimensional obstacle because the number of following directions is infinite. As a result, all the previous sensor-based path-planning algorithms cannot be applicable for a robotic manipulator whose degrees of freedom are over two.

To overcome this problem, we have considered two ways: one is to restrict kinematics of a robotic manipulator, and another is to pick up a part of configuration space systematically. As a former example, a manipulator with two degrees of freedom, or a scalar manipulator without any revolution joint was considered [2], [3], [4]. As a latter example, a major pair of degrees of freedom was selected for a three-degrees-of-freedom manipulator [5], [6], or a 2-D part of config-

uration space of a 6-degrees-of-freedom manipulator was systematically selected by using recursively several dimensional hyper-planes [7], [8], [9]. As a summary, any sensor-based path-planning algorithm for a manipulator with three or more revolution joints has not been presented.

In this paper, we propose a three or more dimensional sensor-based path-planning algorithm $HD - I$ for a robotic manipulator with three or more degrees of freedom. Especially in order to find a leave point around three or more dimensional unknown object, we use a modified version of a *Best - First* algorithm. The reason why we select BF is the similarity to sensor-based path-planning algorithm with the metric condition. That is, both do not consider any cost from a start point to a present point. In other words, both consider only a future cost from a present point to a goal point, which is called **heuristics**.

A new BF used in $HD - I$ differs from an original BF in three following aspects. (1) The new BF starts (begins to input found points into the *OPEN* list) if a robot hits an obstacle, and finishes (initializes *OPEN* and *CLOSED* lists) if the robot leaves the obstacle. (2) A robot traces only boundary of an uncertain obstacle from a hit point to a leave point. (3) A cost of n_a is evaluated by summing length of a shorter route between n_p and n_a and length of the straight segment between n_a and n_g . The nodes n_p , n_a , and n_g are denoted as a present point, a selected point, and a goal point.

The paper is organized as follows: Section 2 describes a robot and its search space. In section 3, we describe algorithms BF and A^* as model-based path-planning algorithms [10], [11], and also explain the algorithm *Class1* as a sensor-based path-planning algorithm [12]. Then, we propose a high dimensional sensor-based path-planning algorithm $HD - I$ for selecting a path from three or more dimensional search space. In section 4, we describe the number N of visiting points and the sum S of moving distances by four types of algorithms in 2-D and 3-D uncertain environments. Finally, section 5 presents a few concluding remarks.

2 A Point Robot and Its Digital Map

In this paper, we adopt a **cell decomposition approach** to path planning, which has a flexibility for representing free shaped objects with three or more dimensions [13]. If the dimension is two, we can plan a path for navigation, and if it is larger than or equals

to three, we can design a path for manipulation.

In this paper, we define a configuration space C_{space} as a collection of cells. First of all, the configuration space C_{space} with m dimensions is built in a joint space $[\theta_1, \dots, \theta_m]$. All the cells are built by dividing an available range of each degree into an arbitrary resolution: $(\theta_1, \dots, \theta_m) / \theta_1 \in [\theta'_1, \theta_1'']$, $\dots$, $\theta_m \in [\theta'_m, \theta_m'']$. The differences $\theta_i'' - \theta_i'$ ($i = 1, \dots, m$) are available ranges, and they are divided by 2^l times. l is given by a human operator, and the arbitrary resolution is determined by $|\theta_i'' - \theta_i'| / 2^l$ ($i = 1, \dots, m$). According to the division, a digital map DM is built by 2^{lm} cells (nodes). For example, 2-D and 3-D digital maps are represented by arrangements $[2^l, 2^l]$ and $[2^l, 2^l, 2^l]$, respectively (Fig.1). A purpose of using a digital map is to express three or more dimensional configuration space. However in this paper, for the simplification of explanation, we basically use a 2-D digital map. Then, all cells (nodes) are classified into two categories C_{free} (*Free* cells) and C_{obs} (*Obs* cells). If and only if a cell (node) does not intersect any configuration space obstacle, the cell (node) is *Free*. Otherwise, the cell (node) is *Obs*. Secondly, a robot is expressed by a cell (node), and is able to move one of neighbor cells (nodes) in the digital map. If the dimension of a configuration space is two and three, eight and twenty-six cells (nodes) are prepared for neighbors, respectively. Two neighbor cells (nodes) are implicitly connected by an edge. Furthermore, the distance (cost) of an edge between horizontal and vertical neighbors is defined by one, and the distance (cost) of an edge between diagonal neighbors is defined by a square route two ($\sqrt{2}$). A total distance between start and goal cells (nodes n_s and n_g) is calculated by summing these neighbor distances.

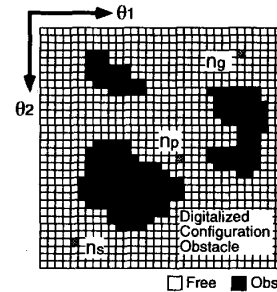


Figure 1 : A cell robot and its 2-D digital map with *Free* and *Obs* cells (nodes).

3 Four Kinds of Algorithms

In this section, we explain three classic path-planning algorithms *BF*, *A** [11], and *Class1* [12]. Then by mixing modified *BF* and *Class1*, we get an algorithm *HD-I* as a 3-D on-line and sensor-based path-planning algorithm.

3.1 An algorithm *BF*

In this paragraph, we explain a classic *BF* algorithm whose search space is a digital map shown in Fig.1. The difference against a state-space graph is as follows: For a node (cell) n_p in the digital map, there are neighbor nodes (cells) $n_i (i = 1, \dots, 3^m - 1)$. The distances (costs $c(n_p, n_i)$) are horizontally and vertically defined by one and diagonally defined by a square route two. Furthermore, since a digital map is a featureless general state-space graph, a robot node (cell) n_p is estimated numerically by a **heuristic evaluation function** $h(n_p)$ which is the *piecewiseL₂* metric between n_p and n_g . As shown in Fig.2, a classic *BF* algorithm cannot select the optimal (shortest) path because it only considers a **future** cost $h(n_p)$ from a present node n_p to a goal node n_g (does not consider a **past** cost $g(n_p)$ from a present node n_p to a start node n_s). In *BF*, the node selected by expansion is the one having the lowest $h()$, and when two paths lead to the same node, the one with the higher $h()$ is discarded. However, because of no consideration of the past cost, the number of nodes visited by *BF*, which are described by a set of gray nodes (cells) in Fig.2, is relatively smaller than the number of nodes investigated by *A**.

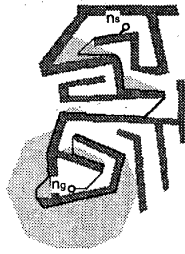


Figure 2 : A selected path and its visited nodes (cells) in *BF*, which are colored by black and gray in a 2-D unknown digital map, respectively.

1. Put a start node n_s on a list called *OPEN* of unexpanded nodes (cells).
2. If *OPEN* is empty, exit with failure; no solution exists.

3. Remove from *OPEN* a node n_p at which $h(n_p)$ is minimum (break ties arbitrary), and place it on a list called *CLOSED* to be used for expanded nodes (cells).

4. Expand node (cell) n_p for selecting all *Free* and unvisited nodes (cells) n_i neighboring n_p , and attach to them pointers back to n_p .

5. If any n_i is a goal node n_g , exit successfully with a shorter path obtained by tracing back the pointers from n_g to n_s .

6. For every neighbor n_i :

(6a) Calculate $h(n_i)$ as the *piecewiseL₂* metric from n_i to n_g .

(6b) If n_i was neither on *OPEN* nor on *CLOSED*, put it on *OPEN*. Assign the newly computed $h(n_i)$ to n_i .

(6c) If n_i already resided on *OPEN* or *CLOSED*, compare the newly computed $h(n_i)$ with the value previously assigned to n_i . If the old value is lower, discard the newly generated node, otherwise, substitute it for the old (n_i now points back to n_p instead of to its previous predecessor). If the matching node n_i resided on *CLOSED*, move it back to *OPEN*.

7. Go to step 2.

An algorithm *BF* always selects a sequence of *Free* nodes (cells) from n_s to n_g since at the worst all *Free* nodes (cells) including n_g are investigated in order of $h()$. The combination search leads a huge set of visited points or moving distances. For example, the set of visited nodes is described as gray area in Fig.2. Therefore, this algorithm cannot be absolutely applied to an on-line or sensor-based path-planning algorithm.

3.2 An algorithm *A**

In this paragraph, we explain a classic *A** algorithm when the search space is a digital map described in Fig.1. The difference against the *BF* algorithm is to consider a past cost $g(n_p)$ from a present node n_p to a start node n_s . An algorithm *A** evaluates a node n_p by summing a **past** cost $g(n_p)$ and a **future** cost $h(n_p)$ as the sum $f(n_p)$, and consequently it always gets an optimal (shortest) path between start and goal nodes. However, the number of nodes visited by *A**, which are described by gray nodes in Fig.3, is relatively larger than the number of nodes investigated by *BF*.

1. Put a start node n_s on *OPEN*.
2. If *OPEN* is empty, exit with failure.
3. Remove from *OPEN* and place on *CLOSED* a node n_p which $f(n_p)$ is minimum (break ties arbitrary).
4. If n_p is a goal node n_g , exit successfully with a shortest (optimal) path obtained by tracing back the

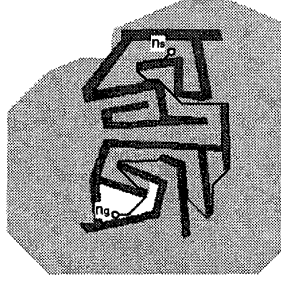


Figure 3 : A selected path and its visited nodes (cells) in A^* , which are colored by black and gray in a 2-D unknown digital map, respectively.

pointers from n_p to n_s .

5. Otherwise, expand n_p for selecting all *Free* and unvisited nodes n_i neighboring n_p , and attach to them pointers back to n_p . For every neighbor node n_i :

(5a) If n_i was not already on *OPEN* or *CLOSED*, estimate $h(n_i)$ by the *piecewise* L_2 metric from n_i to n_g (a lower bound of the shortest route between n_i and n_g), and calculate $f(n_i) = g(n_i) + h(n_i)$ where $g(n_i) = g(n_p) + c(n_p, n_i)$ and $g(n_s) = 0$.

(5b) If n_i was already on *OPEN* or *CLOSED*, direct its pointer along the path yielding the lowest $g(n_i)$.

(5c) If n_i required pointer adjustment and was found on *CLOSED*, remove from *CLOSED* and place on *OPEN* a node n_i .

6. Go to step 2.

Whenever $h(n_p)$ is an **optimistic** estimate of $h^*(n_p)$, an algorithm A^* always returns an optimal (shortest) path. $h^*(n_p)$ is length of an optimal route between n_p and n_g , and $h(n_p)$ is an estimated length of the route. This motivates naming this class of estimates: **Admissible heuristics** [10].

Definition: A heuristic function h is said to be admissible if $h(n_p) \leq h^*(n_p) \forall n_p$.

In A^* , we always get an optimal (shortest) sequence of *Free* nodes from n_s to n_g since $h(n_p)$ is calculated on the straight segment between n_p and n_g , which is a lower bound of their shortest route. It is measured by the *piecewise* L_2 metric. In pursuit of the optimality, A^* needs the number of visited nodes and the sum of moving distances, which are larger than those of *BF*. For example, a larger set of visited nodes is shown as gray region in Fig.3. Therefore, the algorithm cannot be absolutely applicable for an on-line or sensor-based path-planning algorithm.

3.3 The algorithm *Class1*

In this paragraph, we explain the algorithm *Class1* whose search space is a 2-D digital map. In the algorithm, a point robot n_p firstly goes straight to its goal n_g if it does not encounter any obstacle (a set of *Obs* nodes), otherwise, it sets a hit node h_j (a closest node c_j) by n_p . Then, n_p traces a set of *Obs* nodes by the clockwise or counter-clockwise order fixed already. For example, in Fig.4, the counter-clockwise direction is always selected at a hit node h_j . If n_p is closer to n_g than the closest node c_j , c_j is always reset by n_p . In addition to this, if the direction from n_p to n_g is *Free* (that is, a few nodes from n_p to n_g are *Free*), n_p leaves the encountered obstacle (the encountered set of *Obs* nodes) from a leave node l_j . Then, n_p goes straight to n_g again.



Figure 4 : A selected path (a sequence of visited nodes) in *Class1* with the counter-clockwise order, which is colored by black in a 2-D unknown digital map.

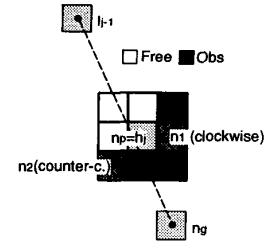


Figure 5 : How to find a neighbor *Free* node n_i around a set of neighbor *Obs* nodes by the clockwise or counter-clockwise order based on the chain coding.

1. Set $j = 1$, and set a robot node n_p and a leave node l_{j-1} and a closest node c_{j-1} by a start node n_s .

2. Select a neighbor node n_i of n_p , which the segment between centers of n_p and n_g intersects in a digital map.

(2a) If n_i is a goal node n_g , exit with success.

(2b) If n_i is *Free*, set n_p by n_i and then be continued, otherwise, set a hit node h_j and a closest node $c_j(j = j + 1)$ by n_p and go to step 3.

3. Select a neighbor *Free* node n_i of n_p by the clockwise or counter-clockwise order based on the chain coding, which is adjacent to a neighbor *Obs* node until one of the following occurs (Fig.5).

(3a) If n_i is a goal node n_g , exit with success.

(3b) If n_i is closer to n_g than the closest node c_j , reset c_j by n_i . Then, if the direction from n_i to n_g is *Free* (that is, a few nodes from n_i to n_g are *Free*), regard n_i as a leave node l_j and return to step 2.

(3c) If n_i equals to the last hit node h_j , exit with failure.

In the algorithm *Class1*, leave and hit nodes (l_j and h_j) monotonously come close to n_g , and consequently n_p arrives at n_g finally if a path (a sequence of *Free* nodes) between n_s and n_g exists in a given map. Otherwise, a point robot n_p automatically recognizes no solution (path) between n_s and n_g in an unknown map by returning the last hit node h_j .

In every previous sensor-based algorithm with the metric condition, e.g. *Class1*, n_p makes only efforts to find a closer node n_i to n_g than c_j . In other words, n_p does not consider any past cost from n_p to n_s . For this reason, similar to *BF*, every sensor-based path-planning algorithm including *Class1* cannot select an optimal path in an unknown map. However, comparing with *A** or *BF*, the number of visited nodes or the sum of moving distances is small enough. For example, a set of visited nodes is described as a sequence of black nodes in Fig.4. In every sensor-based algorithm, the set corresponds to a selected path itself. Therefore, every previous algorithm can be undoubtedly used for an on-line and sensor-based algorithm.

Finally, we should note that all the previous sensor-based path-planning algorithms including *Class1* cannot be applicable for three or more dimensional search space because a point robot n_p has the infinite number of directions to trace an encountered obstacle. To overcome this drawback, we propose a sensor-based path-planning algorithm *HD-I* for searching three or more dimensional space in the next.

3.4 The algorithm *HD-I*

In this paragraph, we focus on *BF* and *Class1* which do not consider a **past** cost $g(n_p)$ from a start node n_s to a present node n_p . Then, we design a three or more dimensional sensor-based path-planning algorithm *HD-I* by mixing modified *BF* and *Class1*. As mentioned previously, the sensor-based path-planning consists of two procedures. One procedure is for a



Figure 6 : A selected path and its visited nodes in *HD-I* with $\alpha = 0.03$, which are colored by black and gray in a 2-D unknown digital map, respectively.

robot to go straight to a goal. Another procedure is for a robot to avoid an uncertain obstacle. In the algorithm *HD-I*, the former procedure and the switching are designed by the algorithm *Class1*, and the latter procedure is designed by the modified *BF*.

Especially in the latter procedure, by using the modified *BF*, a point robot can trace 3-D boundary of an encountered obstacle systematically without generating any limit cycle and overlooking a leave node l_j which is closer to n_g than a closest node c_j . *BF* used in *HD-I* differs from an original *BF* in three following aspects.

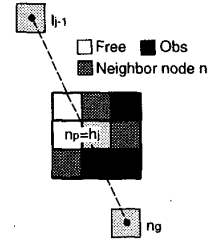


Figure 7 : How to find a neighbor *Free* node n_i of a hit node h_j , which is adjacent to its neighbor *Obs* node in a 2-D digital map.

(1) *HD-I* uses *BF* in order for n_p to trace an encountered obstacle whose dimension is three or more. That is, if a robot node n_p meets a hit node h_j around an uncertain set of *Obs* nodes, *BF* starts (begins to input a found or expanded node into *OPEN* or *CLOSED*), and if n_p reaches a leave node l_j around it, *BF* ends (cleans *OPEN* and *CLOSED*).

(2) *HD-I* uses *BF* in order for n_p to trace boundary of an unknown obstacle, i.e., an uncertain set of *Obs* nodes. First of all, in 2-D and 3-D digital maps, the number of nodes neighboring n_p is denoted by $3^2 - 1$ and $3^3 - 1$, respectively. Secondly, the tracing starts if a robot node n_p finds a *Obs* node as its neighbor n_i on the segment between n_p and n_g . Therefore, n_p is *Free* and n_i is *Obs*. In this case, there is at least

	BF			A*			Class1			HD-I		
	N	S	P	N	S	P	N	S	P	N	S	P
Clockwise	30633	2359953	1308	186384	22497804	922	2965	3155	3155	1307	2466	1365
C-Clockwise	—	—	—	—	—	—	2114	2313	2313	—	—	—

Table 1 : N , S , and P of BF , A^* , $Class1$, and $HD - I$ in a 2-D uncertain digital map.

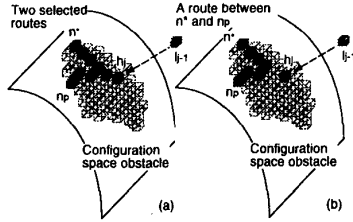


Figure 8 : (a) Two routes are independently selected by tracing back the pointers from n_p and n^* to h_j . (b) A route is obtained by comparing two routes and eliminating their common part and then combining the rest parts.

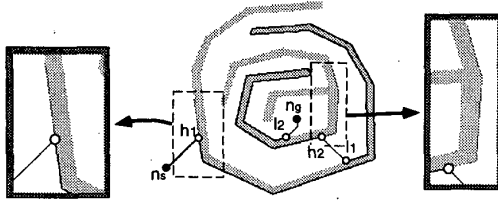


Figure 9 : Trial and error occurs in selection of clockwise and counter-clockwise orders around each hit node h_j when α in $HD - I$ is small enough ($\alpha = 0.03$). A set of visited nodes is described by light gray, and a selected path is colored by dark gray.

one *Free* node neighboring n_p , which neighbors the set of *Obs* nodes. All the *Free* nodes are inserted into *OPEN*.

(3) To select a hopeful node n^* from *OPEN*, we evaluate each node n^* by summing a **moving** cost $m(n^*)$ from n_p to n^* and a **future** cost $h(n^*)$ from n^* to n_g . The future cost is straightforwardly calculated by the *piecewiseL2* metric along the segment between n^* and n_g . On the other hand, the moving cost is defined by multiplying α and the length of a shorter route between n_p and n^* . The route is selected in *BF* as follows: Firstly, two routes are independently obtained by tracing back the pointers from n_p and n^* to h_j (Fig.8(a)). Secondly, a selected route is determined by comparing two routes and eliminating their common part and then combining the rest parts (Fig.8(b)). Finally, the selected route is measured by the *piecewiseL2* metric. The selected route is not always to be an optimal (shortest) route between n_p and n^* . To pick up the optimal route generally, we should use A^* . However as you see, A^* is time con-

suming, and on the other side, the moving cost m will be used as a rough criterion because α is usually less than 0.5. For this reason, in this paper, the shortest route is approximated by a shorter route selected by *BF*.

Comparing with *Class1*, *HD - I* can be straightforwardly applied for an unknown map with three or more dimensions. As a result, we can get a three or more dimensional sensor-based (on-line) path-planning algorithm.

1. Set $j = 1$, and set a robot node n_p and a leave node l_{j-1} and a closest node c_{j-1} by a start node n_s .

2. Select a neighbor node n_i of a robot node n_p , which the segment between n_p and n_g intersects in a digital map.

(2a) If n_i is a goal node n_g , exit with success.

(2b) If n_i is *Free*, set n_p by n_i and then be continued, otherwise, set a hit node h_j and a closest node $c_j(j = j + 1)$ by n_p and go to step 3.

3. Put the node h_j on a list called *OPEN* of un-expanded nodes.

4. If *OPEN* is empty, exit with failure; no path exists.

5. Remove from *OPEN* a node n_p at which q is minimum (break ties arbitrary), and place it on a list called *CLOSED* to be used for expanded nodes.

6. Select each neighbor *Free* node n_i of n_p , which neighbors on the set of *Obs* nodes (Fig.7). For every neighbor n_i :

(6a) If any n_i is a goal node n_g , exit successfully.

(6b) If n_i is closer to n_g than the closest node c_j , reset c_j by n_i and if a few nodes from n_i to n_g are *Free*, regard n_i as a leave node l_j . Then, clear *OPEN* and *CLOSED* and return to step 2.

(6c) If n_i was neither on *OPEN* nor on *CLOSED*, register it to *OPEN* and generate n_i with the pointers back to n_p .

(6d) Evaluate each node n^* in *OPEN* as follows: Firstly, calculate a future cost $h(n^*)$ by the *piecewiseL2* metric along the segment between n^* and n_g . Secondly, calculate a moving cost $m(n^*)$ by the *piecewiseL2* metric along a selected route between n_p and n^* (Fig.8). Finally, assign $q(n^*) = \alpha \times m(n^*) + h(n^*)$ ($0 < \alpha < 1$) to each node n^* in *OPEN*, and then return to step 4.

In the algorithm *HD - I*, leave and hit nodes (l_j and h_j) monotonously converge to a goal node (n_g) in a 2-D or 3-D digital map. In addition, n_p always finds an adequate leave node l_j around an encountered set of *Obs* nodes as long as a sequence of *Free* nodes from n_s to n_g exists in a given digital map. For this reason, if such a sequence exists, n_p certainly arrives at

BF			A*			HD-I		
N	S	P	N	S	P	N	S	P
1162	10563	187	77058	1528502	138	1082	1930	777

Table 2 : N , S , and P of BF , A^* , and $HD-I$ in a 3-D uncertain digital map.

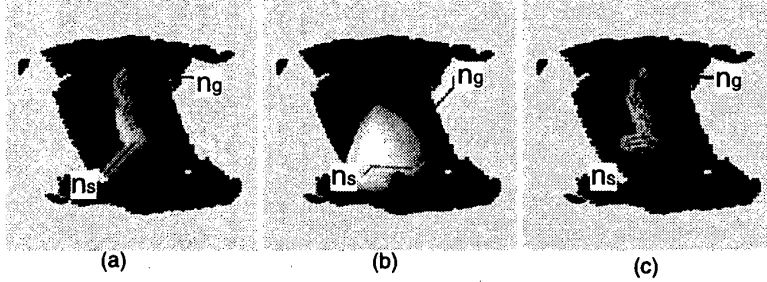


Figure 10 : (a), (b), (c) Selected paths and their sets of visited nodes in BF , A^* , and $HD-I$. Each path and set are colored by dark and light gray in a 3-D uncertain digital map, respectively.

n_g , otherwise, n_p sees no path since $OPEN$ becomes empty.

4 Simulation Results

In this section, we compare the number N of visited nodes, the sum S of moving distances, and the length P of a selected path in a 2-D digital map described in Fig.2, 3, 4 and 6 by BF , A^* , $Class1$ and $HD-I$, respectively (Table 1). Firstly, the 2-D digital map is given by $[2^9, 2^9]$ ($l=9, m=2$), and includes 262144 nodes totally, 32368 *Obs* nodes, and 229776 *Free* nodes. Now, our questions concerning to path-planning are how many nodes a point robot visits in the map and how long the robot moves in it. If the number N is small enough, the algorithm is regarded as the on-line algorithm, otherwise, it is considered as the off-line algorithm. In addition, if and only if the value S is small enough, the algorithm is regarded as the sensor-based algorithm.

As shown in Table 1, N of BF or A^* is extremely larger than N of $Class1$ or $HD-I$ with $\alpha = 0.03$, i.e., $N_{A^*} \gg N_{BF} \gg N_{Class1} > N_{HD-I}$. Therefore, the former two are off-line, and the latter two are on-line. Here, the inequality $N_{Class1} > N_{HD-I}$ is kept. The reason is that a robot supervised by $Class1$ fixes its direction following an encountered obstacle and consequently sometimes loses the way to its destination in a complicated uncertain environment shown in Fig.4. On the other hand, a robot supervised by $HD-I$ with a small α always selects an adequate direction at a hit node h_j by trial and error concerning to selection of clockwise or counter-clockwise direction (Fig.6 and 9). Especially in Fig.9, due to the trial and error, a point robot locally changes clockwise and counter-clockwise directions around two hit nodes in order to find an

adequate direction globally. In conclusion, a set of visited nodes widely spreads out around boundary of an unknown obstacle at hit nodes, which is colored by light gray in Fig.9. As a result, even though an environment has complex shape, the algorithm $HD-I$ whose α is small enough ($\alpha = 0.03$ in Fig.6, 9, Table1) is the best in the on-line path-planning.

For the simplification, S of BF and A^* is given by a lower bound of the true moving distances. The reason is that each distance between n_p and n^* is easily calculated as the distance of their straight segment. The distance is always smaller than or equals to the distance of their shortest route. This property is true even in the *piecewise* L_2 metric. As contrasted with this, S of $HD-I$ is calculated as an upper bound of the true moving distance. The reason is that each distance between n_p and n^* is calculated as the *piecewise* L_2 metric along a satisfactory route selected in $HD-I$, which was explained in the last section (Fig.8). The route is always longer than or equals to the shortest route between n_p and n^* . In addition, S of $Class1$ is calculated as the true moving distance.

Nevertheless, S of BF or A^* is extremely larger than S of $Class1$ or $HD-I$, i.e., $S_{A^*} > S_{BF} \gg S_{Class1} \approx S_{HD-I}$. Therefore, we can see that the algorithms $Class1$ and $HD-I$ are better ones as the 2-D sensor-based path-planning algorithm. Finally, as shown in Table 1, P obtained by A^* is the shortest, and P selected by BF , $Class1$, or $HD-I$ nearly equals, i.e., $P_{A^*} < P_{BF} \approx P_{Class1} \approx P_{HD-I}$. For this reason, A^* is the best algorithm in the 2-D off-line (model-based) path-planning.

Secondly, we compare the number N of visited nodes, the sum S of moving distances, and the length P of a selected path in a 3-D digital map (Table 2). Selected paths and visited sets by BF , A^* , and $HD-I$ with $\alpha = 0.5$ are described in Fig.10(a), (b), and (c).

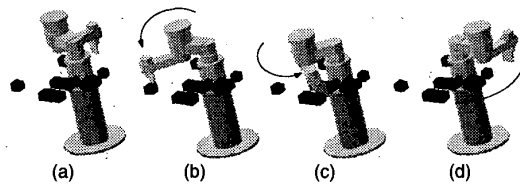


Figure 11 : (a), (b), (c), (d) A sequence of manipulator motions made in $HD - I$.

Firstly, the 3-D digital map is given by $[2^7, 2^7, 2^7]$ ($l=7$, $m=3$), and includes 2097152 nodes totally, 396579 *Obs* nodes, and 1700573 *Free* nodes.

As shown in Table 2, N of A^* is extremely larger than N of BF or $HD - I$, i.e., $N_{A^*} \gg N_{BF} > N_{HD-I}$. Therefore, the former algorithm is off-line, and the latter two algorithms are on-line. Shape of the unknown obstacle is not so complex, and therefore BF is not so bad as an on-line algorithm, which sequentially picks up a closest node to n_g . On the other hand, S of A^* or BF is extremely larger than S of $HD - I$, i.e., $S_{A^*} \gg S_{BF} \gg S_{HD-I}$. Therefore, we can see that the algorithm $HD - I$ with a large α is the best algorithm in the 3-D sensor-based path-planning (Table 2). Especially in an unknown map whose obstacles are simple shape, a robot supervised by $HD - I$ with a large α selects a satisfactory path globally without many trial and error at each hit point h_j locally ($\alpha = 0.5$ in Fig.10(c) and Table 2). As a result, the algorithm $HD - I$ with a large α is the best sensor-based path-planning algorithm. Finally, the length P given by A^* or BF is the shortest or shorter, i.e., $P_{A^*} \approx P_{BF} \ll P_{HD-I}$. For this reason, A^* and BF are regarded as the 3-D off-line path-planning algorithm in such a map with simple shape.

Finally, the used 3-D map is given as a configuration space of a three-degrees-of-freedom manipulator. Therefore, a selected path corresponds to a sequence of manipulator motions described in Fig.11, which is generated by $HD - I$. All programs are running in a personal computer Gateway GP6-400 (Windows 98 with Open GL, Pentium II 400MHz, 256MB main memory).

5 Conclusions

In this paper, we propose a sensor-based path-planning algorithm for investigating an uncertain environment whose dimensions are three or more. The key idea of our new algorithm is mixing two modified algorithms BF and $Class1$. Especially based on a modified BF algorithm, a point robot is flexibly able to avoid boundary of an unknown obstacle whose directions are over two. Therefore, the proposed algorithm can be used for manipulation or assembly by a robotic manipulator. By several simulation results, usefulness and efficiency of our algorithm are ascertained.

References

- [1] H.Noborio, "On a sensor-based navigation for a mobile robot," *Journal of Robotics Mechatronics*, Vol.8, No.1, pp.2-14, 1996.
- [2] V.J.Lumelsky, "Effect of Kinematics on motion planning for planar robot arms moving amidst unknown obstacles," *IEEE Journal of Robotics and Automation*, Vol.3, No.3, pp.207-223, June 1987.
- [3] V.J.Lumelsky and K.Sun, "A unified methodology for motion planning with uncertainty for 2D and 3D two-link robot arm manipulators," *The International Journal of Robotics Research*, Vol.9, No.5, pp.89-104, October, 1990.
- [4] K.Sun and V.J.Lumelsky, "Path planning among unknown obstacles: The case of a three-dimensional cartesian arm," *IEEE Trans. on Robotics and Automation*, Vol.8, No.6, pp.776-786, December, 1992.
- [5] E.Cheung and V.J.Lumelsky, "Real-time path planning procedure for a whole-sensitive robot arm manipulator," *Robotica*, Vol.10, pp.339-349, 1992.
- [6] V.J.Lumelsky and E.Cheung, "Real-time collision avoidance in teleoperated whole-sensitive robot arm manipulators," *IEEE Trans. on System, Man, and Cybernetics*, Vol.23, No.1, pp.194-203, January/February, 1993.
- [7] H.Noborio, "A hyperplane-based path planning algorithm for a four-degrees-of-freedom robotic manipulator in an uncertain workspace," *Proc. of the USA-Japan Symposium on Flexible Automation*, pp.95-98, July 1992.
- [8] H.Noborio, T.Hamaguchi, and T.Yamamoto, "Experiments with a 3DD robotic manipulator operating by a sensor-based assembly," *Proc. of the IEEE International Symposium on Assembly and Task Planning*, pp.249-256, August 1997.
- [9] H.Noborio, M.Kadowaki, and K.Urakawa, "A near-optimal sensor-based motion-planning algorithm for parts mating," *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, pp.510-517, October 1998.
- [10] N.Nillson, "Principle of artificial intelligence", *Tioga Publishing Co.*, 1980.
- [11] J. Pearl, "Heuristics", *Addison Wesley*, October 1985.
- [12] H.Noborio, "A sufficient condition for designing a family of sensor-based deadlock-free path-planning algorithms," *Journal of Advanced Robotics*, Vol.7, No.5, pp.413-433, January 1993.
- [13] J.-C. Latombe, "Robot motion planning", *Chapter 6: Approximate Cell Decomposition*, *Kluwer Academic Publishers*, 1991.